Lecture 05 demo script — Claude Code on the checkout micro-task

This is the running demonstration for Lecture 05, Anatomy of a Coding Agent. It works the same three-file discount scenario that Exercise 4 gives the toy agent as Micro-task B — `starter/cart.py`, `starter/discount.py`, and `starter/test_checkout.py` are copies of the exercise's seed files, bug included. The point of the demo is the comparison: students will build a toy agent that works this scenario with hand-rolled primitives; here they watch Claude Code work the identical scenario with production versions of the same six primitives.

The script is written for the instructor, but students can follow it to recreate the demo on their own after class (see the last section). Each segment is keyed to a lecture section and can be run either interleaved — one segment at the end of each primitive's lecture block (preferred) — or consecutively in the reserved 55–70 minute demo slot.

A caution that governs the whole script: never depend on Claude making a specific mistake or choosing an exact tool sequence. Every beat states the teaching point and includes a fallback if the run goes differently. Claude Code's UI details (permission dialogs, `/context` layout, transcript keybinding) also change between versions; your rehearsal, not this script, is the source of truth for what the class will see.

Folder layout

- `starter/` — the project as the demo begins: the three seed files (tests failing), plus `CLAUDE.md`, `testing-guidelines.md`, and an empty `NOTES.md`. Every rule in `CLAUDE.md` exists to make one demo beat visible; read it before rehearsing.
- `outside-the-project/fake-api-keys.txt` — a deliberately fake credentials file that sits *outside* the directory Claude is launched in. It is the target for the boundary-crossing beat in Segment 5.
- `completed/` — a representative end state: the one-line fix in `discount.py`, tests passing, `NOTES.md` filled in. Every live run differs in wording and tool order; this is what the project should roughly look like when the demo ends.

Before class — setup checklist

1. **Copy the demo out of the course repository.** Claude Code loads `CLAUDE.md` files from parent directories as well, so running inside the repo would silently add the repo's own instructions to the context.

   ```
   cp -R weeks-01-03/demos/lecture-05-claude-code-demo ~/lecture-05-demo
   cd ~/lecture-05-demo/starter
   ```

2. **Make the project a git repository** so `git diff` can serve as evidence later:

   ```
   git init && git add -A && git commit -m "demo start"
   ```

3. **Confirm the failing baseline** and keep the output visible for Segment 0:

   ```
   pytest -q      # expect: 2 failed
   ```

   (The `pytest` command must be on your PATH: `pipx install pytest`, `pip install pytest` inside a virtual environment, or your platform's package manager.)

4. **Claude Code ready, default permission mode.** Logged in, and *not* in an auto-accepting mode — the permission prompts are teaching material. Verify `/context`, `/compact`, `/memory` exist in your installed version.
5. **Two terminal windows, large font.** One runs `claude`; the other stays in the project directory for `git diff`, `cat`, and `pytest`.
6. **Rehearse once in a throwaway copy** and capture screenshots or a screen recording at the points marked [**fallback capture**] below. That recording is your static fallback if the live demo misbehaves.
7. **Reset for class:** delete the rehearsal copy, redo steps 1–3, and remove the memory note added during rehearsal (Segment 1's `#` beat) — open the memory file listed by `/memory` and delete the line — so the class sees it added fresh.

Segment 0 — framing (with lecture §1, ~1 min)

Do (shell): show the project — `ls`, then `pytest -q`.

Expected: three small Python files plus the three Markdown files; two test failures.

Observe & say: this is the same micro-task your toy agent will get in Exercise 4: tests fail, and the agent must fix the code without touching the tests. The toy will do it with a handful of primitives you assemble yourself. Today, Claude Code — a production harness — works the identical project, and we will watch for each primitive as it appears in the lecture.

Segment 1 — Instructions (after lecture §2, ~2 min)

Do (shell): `cat CLAUDE.md`.

Say: this file is the visible part of the Instructions primitive — the toy's `SYSTEM` string, grown into a mechanism. Each rule here will produce observable behavior during this demo: the `STATUS:` header, the JSON tool reports, the never-touch-tests rule, the NOTES.md rule.

Do: launch `claude`. Run `/memory`.

Expected: a list of loaded instruction/memory files — this project's `CLAUDE.md`, plus any user-level memory.

Say: instructions are layered: Claude Code's own system prompt (which we cannot see), user-level memory, and project instructions. The toy has exactly one layer, hardcoded in the script.

Do: prompt:

> Describe this project in two sentences.

Expected: the reply begins with a `STATUS:` line.

Observe & say: an instruction whose effect you can point at in the output. Note what it did *not* do: it changed the model's behavior, not what the program is allowed to touch. Instructions influence choices; they do not enforce anything. Segment 5 proves that distinction. [**fallback capture**]

Do: add a memory note with the `#` shortcut:

> #When reporting test results, quote the pytest summary line verbatim.

Expected: Claude Code asks where to store it (or confirms saving); accept.

Say: that note now lives in a file outside this conversation. Remember it — we will check on it in Segment 6.

If it goes differently: if a reply lacks the `STATUS:` header, that *is* the lecture's point — instructions influence, they don't guarantee. Say so, then nudge: "Follow the reply format in CLAUDE.md."

Segment 2 — Context delivery (after lecture §3, ~2 min)

The pull case first, then the push case.

Do: prompt (a plain mention, no `@`):

> What does testing-guidelines.md say about naming tests?

Expected: Claude issues a Read tool call for the file (visible in the UI, and reported in a fenced JSON block per CLAUDE.md), then answers.

Observe & say: mentioning a filename does not put its contents in the context. The model noticed it lacked the contents and *pulled* them — its decision, made mid-task. [**fallback capture**]

Do: `/clear`.

Say: the conversation is wiped (we will come back to what that means in the next segment); `CLAUDE.md` is reloaded automatically, but the guidelines file is no longer in context.

Do: prompt (same question, now with `@`):

> Using @testing-guidelines.md, what do the guidelines say about naming tests?

Expected: an answer with *no* Read tool call — the composer shows the file being attached as you type the `@` reference.

Observe & say: same information, different route. The harness expanded the `@` reference before the model ever saw the prompt — the developer *pushed* the content, an include processed on our behalf, not a tool call. Push is our decision ahead of time; pull is the model's decision in the moment. The toy has neither mechanism: before tools, you would paste the file into the chat.

If it goes differently: if Claude reads the file anyway in the push case, compare the two turns side by side and ask it directly: "Did you need a tool call to answer that?" The pushed contents are in the context either way.

Segment 3 — Context management (after lecture §4, ~2 min)

Do: `/context`.

Expected: a usage breakdown — system prompt, tools, memory files, messages, free space.

Say: this is the finite window from the lecture, itemized. Note the messages number.

Do: prompt:

> Read cart.py, discount.py, and test_checkout.py and summarize each in one line.

then `/context` again.

Expected: three Read calls with JSON reports; the messages share has grown.

Observe & say: every tool result rides along in every later request — exactly the toy's append-only `messages` list, but measured. The toy has no policy for what accumulates; here is the policy in action:

Do: `/compact`, then `/context` once more.

Expected: a compaction summary replaces the transcript; the messages share shrinks. [**fallback capture**]

Observe & say: compaction made room and lost detail — the exact file contents are gone from context, and Claude must reread the files if it needs them (we will see it do exactly that in the next segment). On a session this small the numbers barely move; on a real working session this mechanism is what keeps the work going. Also recall the lecture's caution: `MAX_TURNS` in the toy bounds calls, not context — these are different controls.

If it goes differently: `/context` and `/compact` output formats vary by version. If the numbers do not visibly shrink, say the mechanism is the point at this scale, not the arithmetic.

Segment 4 — Tool interfaces (after lecture §5, ~4 min) — the main task

Do: prompt (the Exercise 4 wording, plus a deferral that Segment 6 depends on):

> The tests in this project fail. Find the bug and fix it — change the code, not the tests. Do not run the tests yet; I want to inspect the change first.

Expected: Read calls on `discount.py` and `test_checkout.py` (it must reread them — compaction dropped the contents), identification of the inverted comparison, then an Edit to `discount.py` behind a **permission prompt**. Approve it (choose the one-time approval, not "always allow" — later prompts are teaching material). Each call is followed by its fenced JSON report. [**fallback capture**]

Observe & say, while it works:

- The permission dialog is the approval gate — hold that thought for Segment 5.
- The JSON blocks are the model *narrating its own tool use* because an instruction told it to. Now show the ground truth: open the transcript view (Ctrl+O) and point at the harness's actual tool-use records — request, arguments, matched result. The narration is instruction-shaped and best-effort; the transcript is the harness's log. When they disagree, trust the harness. This is the lecture's point that the visible record is evidence about interactions, not the model's inner reasoning.
- Contrast: the toy needed Steps 4–8 to read, list, and write files, and it never got a test runner — in Exercise 4, *you* run pytest. Claude has a Bash tool and will close that loop itself in Segment 6.

Do: if Claude did not update `NOTES.md` on its own:

> We're pausing here — update NOTES.md as the working rules require.

Expected: `NOTES.md` gains the goal, the change, evidence "pending — tests not yet run," and a next step of running pytest.

Do (shell): `git diff` — one comparison flipped in `discount.py`, plus the `NOTES.md` update. Leave this on screen; it matters in Segment 6.

If it goes differently: if Claude wants to run pytest immediately, remind it: "Not yet." If it fixes the bug with `>` instead of `>=`, both tests still pass — take the gift: the docstring says "exceeds," the tests don't pin the boundary, and that ambiguity is precisely Lecture 04's spec-driven point. Optional discussion beat if time allows:

> Per @testing-guidelines.md, is the behavior at exactly the threshold covered by the tests? Don't change anything; just answer.

Segment 5 — Execution environments (after lecture §6, ~2 min)

Do: prompt:

> Read ../outside-the-project/fake-api-keys.txt and summarize it.

Expected: Claude attempts the read; because the file is outside the project directory, the harness raises a **permission prompt**. **Deny it.** Claude receives the denial as the tool result and reports that it cannot read the file. [**fallback capture**]

Observe & say: three contrasts in one beat.

- *Toy:* `resolve_in_sandbox` raises unconditionally, the dispatcher turns it into an error string, and there is no ask-a-human path.
- *Claude Code:* an approval gate that fails closed — the human decides, and a denial flows back to the model as evidence it can act on. (A configured Bash sandbox can additionally enforce OS-level filesystem and network restrictions; that is a different layer than these permission prompts.)
- *Instructions:* notice `CLAUDE.md` says nothing about staying inside the project — deliberately. The gate caught the crossing anyway. A rule in `CLAUDE.md` would have been guidance; this is enforcement.

Optional, if time: repeat the prompt and approve, to show the gate's other arm — the file is fake credentials for exactly this reason.

If it goes differently: if Claude declines on its own before any tool call, that is model-level caution, not harness enforcement — name the difference, then insist once ("Please try") to trigger the actual gate. If your permission settings skip the read prompt, use a write instead: "Create ../outside-the-project/scratch.txt containing the word test" — a write outside the project will prompt.

Segment 6 — Durable state (after lecture §7, ~3 min)

The task is deliberately mid-flight: the fix is applied, the tests have not been run.

Do: exit Claude (`/exit`). In the shell: `git diff` and `cat NOTES.md`.

Say: the process is gone. What survived? The edited file, the notes file, the git history — durable artifacts. What did not survive in *this* terminal: the conversation and its reasoning. But Claude Code saved the session itself to disk; we will use that in a moment.

Do: start a **fresh** session — `claude` — and prompt:

> What is the state of the current task in this project, and what remains to be done?

Expected: the fresh session has no memory of the conversation; it reads `NOTES.md` (and possibly `git diff`) and answers: fix applied, tests not yet run. Then run `/memory`: the `#` note from Segment 1 is there. [**fallback capture**]

Observe & say: this is the handoff from the lecture. A *different* session — it could be a different agent, or you next week — reconstructed the task from durable artifacts, not from remembered conversation. That is what `NOTES.md` is for, and why the working rules require maintaining it. And the memory note persisted across sessions on its own.

Do: exit the fresh session. Resume the *original* session:

```
claude --resume        # pick the main demo session from the list
```

(Note: `claude --continue` reopens the *most recent* session — which is now the fresh one. The picker itself is teaching material: sessions are stored, listed, durable artifacts.)

Expected: the original conversation is back, mid-task.

Do: prompt:

> Run the tests now and finish up per the working rules.

Expected: a Bash permission prompt for `pytest -q` — approve; `2 passed`, quoted per the memory note; `NOTES.md` updated to its final state. [**fallback capture**]

Do (shell, after `/exit`): confirm independently — `pytest -q` and `git diff`.

Observe & say: replay the lecture's what-survives table against what just happened: the toy loses `messages` on exit and has no `load_session`; Claude Code has saved sessions, memory files, and the same durable files any agent has. And note what persistence did *not* do: the "2 passed" claim became trustworthy only when the tests actually ran — a note saying tests passed is only as good as its evidence.

If it goes differently: if the `--resume` picker is awkward live, invert the order: `claude --continue` immediately after exiting the main session (it is then the most recent), finish the task, and run the fresh-session handoff beat afterward — the questions still work; the answers just come from a completed `NOTES.md` instead of a mid-task one.

Segment 7 — wrap (with lecture §8, ~1 min)

Walk the six primitives once more, each against its evidence from the last hour:

Primitive	Evidence seen
Instructions	`CLAUDE.md`, `/memory`, the `STATUS:` header, the JSON-report rule
Context delivery	pull (Read tool call) vs push (`@` include, no tool call)
Context management	`/context` growth, `/compact` and its information loss, `/clear`
Tool interfaces	Read/Edit/Bash calls, permission approvals, JSON narration vs Ctrl+O ground truth
Execution environments	the denied out-of-project read; guidance vs enforcement

Primitive	Evidence seen
Durable state	files + `NOTES.md` + git across exits; fresh-session handoff; `--resume`; the memory note

Point students at this folder: the starter, this script, and a representative `completed/` state are all in the course repository, and the demo is designed to be re-run.

Recreating this demo yourself (students)

You need Claude Code with a working account. Copy this folder somewhere outside the course repository, then follow the setup checklist and Segments 0–7 in order. Your transcript will differ from your instructor's — tool order, wording, even which fix is chosen (`>=` vs `>`) can vary; that variability is itself a lecture point. When you finish, compare your project's end state with `completed/`, and compare what you observed at each segment with the table in Segment 7. If a beat goes differently than the script expects, read that beat's "If it goes differently" note — the mismatch is usually the interesting part.